C++ – enable_shared_from_this – 避免用t...

使用场景

- 当一个类对象被shared_ptr管理，且在这个类的成员函数里需要把当前类对象作为参数传给其他函数时，就需要传递一个指向自身的shared_ptr。

错误写法（直接传递this指针或包装this指针为shared_ptr）

- **如果用以下方法返回shared_ptr，会出现裸指针初始化多个shared_ptr的问题，导致多次析构！**

```
#include<iostream>
#include<memory>

class Widget;

\nvoid Process(const std::shared_ptr<Widget>& sp) {

}

class Widget {
public:\n    std::shared_ptr<Widget> GetWidget() {
        return std::shared_ptr<Widget>(this);
    }

\n    void Invoke() {\n        Process(std::shared_ptr<Widget>(this));
    }
};

\nint main() {

    {
        auto sp1 = std::make_shared<Widget>();
        //auto sp2= sp1->GetWidget(); //Error, 两个共享指针指向同一个裸指针
    }

    std::cin.get();
    return 0;
}

    std::cin.get();
    return 0;
```

B站：精神浪子_码农

```
}
```

正确写法

- **步骤1：用户类必须公有继承于标准库类enable_shared_from_this**
  - 标准库类enable_shared_from_this会使用**奇异递归设计模板（CRTP）**构建std::weak_ptr<T>，用来跟踪管理当前对象的std::shared_ptr，提供shared_from_this函数来将内部的的std::weak_ptr提升为std::shared_ptr

```
#include <memory>  // 依赖 weak_ptr/shared_ptr

namespace std {

template <class T> //CRTP
class enable_shared_from_this {
protected:
    enable_shared_from_this() noexcept = default;
    ~enable_shared_from_this() = default;

public:\n    shared_ptr<T> shared_from_this() {
        return shared_ptr<T>(_Wptr);
    }
    \n    shared_ptr<const T> shared_from_this() const {
        return shared_ptr<const T>(_Wptr);
    }

private:
    weak_ptr<T> _Wptr;

    template <class _Other>
    friend class shared_ptr;
};

} // namespace std
} // namespace std
```

  - 上述定义可以看到，构造函数并没有对内部的的std::weak_ptr进行初始化，那什么时候初始化呢，其实是在标准库的std::share_ptr的构造过程中对其初始化。

```
namespace std {
template<typename _Tp>
class shared_ptr
{
public:
    // 构造 shared_ptr（管理裸指针）
    template<typename _Yp>
    explicit shared_ptr(_Yp* __p)
        : _M_ptr(__p), _M_refcount(__p)
```

B站：精神浪子_码农

```
    {
        // 👉 关键：调用辅助函数，初始化 enable_shared_from_this 内部的 weak_ptr
        __enable_shared_from_this(this, __p);
    }

private:
    _Tp* _M_ptr;              // 对象指针
    __RefCount _M_refcount;   // 控制块

    // 👉 核心辅助函数：检测并初始化 enable_shared_from_this
    template<typename _Yp>
    void __enable_shared_from_this(shared_ptr<_Tp>* __this, _Yp* __p)
    {
        // 检测：__p 是否继承自 enable_shared_from_this<_Tp>
        using __base_type = enable_shared_from_this<_Tp>;

        // 若是，自动把当前 shared_ptr 赋值给内部 weak_ptr
        if constexpr (is_convertible_v<_Yp*, __base_type*>) {
            __p->_M_weak_this = *__this; // 👉 初始化！
        }
    }
};
} // namespace std
```

- **步骤2：用户类创建的对象必须使用shared_ptr形式使用，不能单独定义使用！**
  - 上述的标准库定义就决定了如下的使用方式是错误的，**因为如下的w1和p都没有用shared_ptr构建对象，从而内部的的std::weak_ptr没有得到初始化，会引发标注库错误（std::bad_weak_ptr）**

```
#include<iostream>
#include<memory>

class Widget;

\nvoid Process(const std::shared_ptr<Widget>& sp) {

}

class Widget : public std::enable_shared_from_this<Widget> {
public:\n    std::shared_ptr<Widget> GetWidget() {\n        return shared_from_this();
    }

\n    void Invoke() {\n        Process(shared_from_this());
    }
};
```

B站：清神浪子码农

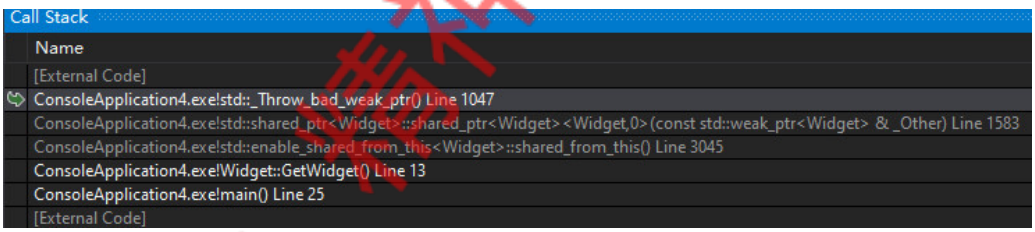
```
\nint main() {
    {
        //Error, std::bad_weak_ptr
        Widget w1;
        std::shared_ptr<Widget> w2 = w1.GetWidget();// &w1
        std::cout << "ref count: " << w2.use_count() << std::endl;

        //Error, std::bad_weak_ptr
        Widget* p = new Widget();
        std::shared_ptr<Widget> w3 = p->GetWidget();
    }
    //terminate called after throwing an instance of 'std::bad_weak_ptr' what():  bad_weak_ptr

    std::cin.get();
    return 0;
}

    //terminate called after throwing an instance of 'std::bad_weak_ptr' what():  bad_weak_ptr

    std::cin.get();
    return 0;
}
```



- **步骤3：将构造函数private禁止创建栈对象，使用静态工厂函数创建shared指针**
  - 既然用户类对象只能**使用shared_ptr指针的形式来使用，所以最安全的做法是：将构造函数private禁止创建栈对象、使用静态工厂函数确保创建shared指针。**
  - 使得用户无法使用值语义对象(error C2248: ’Widget::Widget’: cannot access private member declared in class ’Widget’)，从源头杜绝错误使用方式。

```
#include<iostream>
#include<memory>

class Widget;

\nvoid Process(const std::shared_ptr<Widget>& sp) {

}

```

B站：青神浪子_码农

```
class Widget : public std::enable_shared_from_this<Widget> {
public:\n    std::shared_ptr<Widget> GetWidget() {\n        return shared_from_this();
    }

\n    void Invoke() {\n        Process(shared_from_this());
    }

\n    static std::shared_ptr<Widget> Create() {
        return std::shared_ptr<Widget>(new Widget());
    }

private:\n    Widget() = default;
};

\nint main() {
    {
        //Error C2248: 'Widget::Widget': cannot access private member declared in class 'Widget'
        // Widget w1;
        // std::shared_ptr<Widget> w2=w1.GetWidget();// &w1
        // std::cout<<"ref count: "<<w2.use_count()<<std::endl;

        // Widget* p= new Widget();// w1;
        // std::shared_ptr<Widget> w2 = p->GetWidget();// &w1
    }

    {
        std::shared_ptr<Widget> sp1=Widget::Create();

        auto sp2=sp1->GetWidget();
        std::cout<<"ref count: "<<sp1.use_count()<<std::endl;
        std::cout<<"ref count: "<<sp2.use_count()<<std::endl;

        std::cout<< sizeof(Widget)<<std::endl;
    }


    std::cin.get();
    return 0;
}

/*
ref count: 2
ref count: 2
16
*/

```

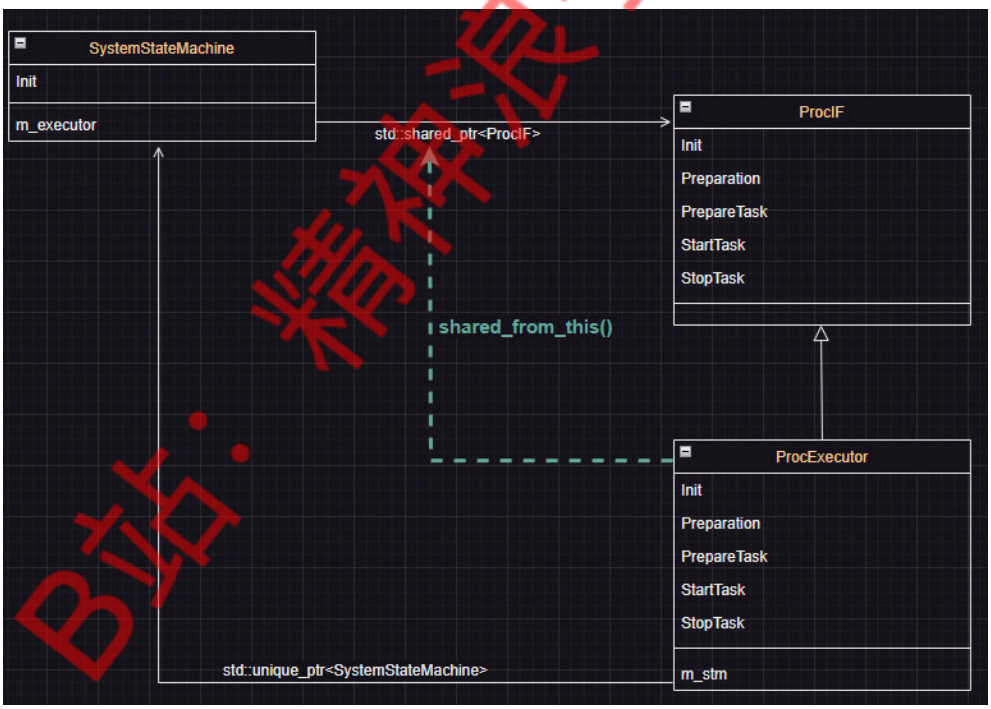
B站:精神浪子_码农

```
}

/*
ref count: 2
ref count: 2
16
*/
```

工程应用

- SystemStateMachine为状态机类，包含一个状态切换时需要调用的执行类的对象的shared_ptr指针m_executor，用于状态切换时调用m_executor的某个执行函数，m_executor的对象类型是基类ProcIF类型。
- 执行类ProcExecutor内部又有一个SystemStateMachine的对象m_stm，即状态机对象m_stm是由ProcExecutor对象创建的。而创建时传入的参数就是执行类ProcExecutor这个对象的智能指针（**子类ProcExecutor的shared_ptr可以直接转换为基类ProcIF的shared_ptr**）这个参数用来初始化SystemStateMachine对象的m_executor，UML图如下：



- 所以如果要达到这个目的，就需要真正的执行类ProcExecutor对象将自身包装为shared_ptr传给SystemStateMachine状态机对象，这种场景需要使用标准库的enable_shared_from_this，按照上述正确写法中的步骤进行：
  - **步骤1：执行类ProcExecutor公有继承于enable_shared_from_this**
  - **步骤2：执行类ProcExecutor对象必须使用shared_ptr形式使用，及调用类的静态成员函数Create函数创建。**
  - **步骤3：执行类ProcExecutor对象的构造函数私有化，禁止创建其栈对象。**

B站：精神浪子码农

- 需要执行类ProcExecutor对象的shared_ptr就调用Instance()函数，实际上调用的就是标准库shared_from_this()函数。
- 这个示例工程大致的执行过程为：
  - 调用ProcExecutor类的静态成员函数Create创建ProcExecutor对象。
  - 调用ProcExecutor对象的Init函数就行初始化，这个函数里创建SystemStateMachine状态机对象，同时把ProcExecutor对象包装为shared_ptr（enable_shared_from_this）传递给SystemStateMachine状态机对象，并调用状态机对象的Init函数。
  - 调用ProcExecutor对象的Preparation函数将状态机切换到Preparation状态，这里为了简单说明问题，就直接调用了SystemStateMachine状态机对象里的m_executor（即ProcExecutor对象）的PrepareTask函数。
  - 实际情况下，会在调用状态机对象的Init函数时候将状态切换和ProcExecutor对象的某些成员函数绑定，然后在切换状态机状态时自动调用这些函数里面的某一个。

```
#include <iostream>
#include <memory>
#include <any>

enum class SystemAction {
    RESET = 0,
    PREPARETION,
    EXECUTE,
    FINISH,
    STOP,
    MAX
};

class ProcIF
{
public:
    virtual ~ProcIF() = default;\n  virtual bool PrepareTask(const std::any&, const std::any&) = 0;
};
using ProcPtr = std::shared_ptr<ProcIF>;


class SystemStateMachine
{
public:\n   SystemStateMachine(ProcPtr executor) : m_executor{ executor } {}

\n  void Init() {
        std::cout << "STM Init!" << std::endl;
        //...
    }
```

B站：精神浪子二码农

```
\n  void Action(SystemAction action, const std::any& arg1, const std::any& arg2) {\n      if (action == SystemAction::PREPARETION) {
            m_executor->PrepareTask(arg1, arg2);
        }
        //....
    }

private:
    ProcPtr m_executor;
};

class ProcExecutor : public ProcIF, public std::enable_shared_from_this<ProcExecutor>
{
public:

\n  static std::shared_ptr<ProcExecutor> Create() {
        return std::shared_ptr<ProcExecutor>(new ProcExecutor());
    }

\n  std::shared_ptr<ProcIF> Instance() { return shared_from_this(); }

\n  void Init()
    {
        m_stm = std::make_unique<SystemStateMachine>(Instance());\n if (m_stm != nullptr) { m_stm->Init(); }
    }

\n  void Preparation()
    {
        m_stm->Action(SystemAction::PREPARETION, 11, std::string("Test"));
    }

\n  bool PrepareTask(const std::any& arg1, const std::any& arg2)    override {
        std::cout << __FUNCTION__ << std::endl;
        //...
        return true;
    }

private:\n  ProcExecutor() = default;
    std::unique_ptr<SystemStateMachine> m_stm;
};

//...
```

B站：精神浪子-码农

```
\nint main(int argc, char** argv)
{
    auto executor = ProcExecutor::Create();
    executor->Init();
    executor->Preparation();
    return 0;
}

/*
STM Init!
ProcExecutor::PrepareTask
*/

int main(int argc, char** argv)
{
    auto executor = ProcExecutor::Create();
    executor->Init();
    executor->Preparation();
    return 0;
}

/*
STM Init!
ProcExecutor::PrepareTask
*/
```

思考点

- 一般来说，使用make_shared比使用new更安全更高效，那为什么在使用enable_shared_from_this的场景里，创建类的shared_ptr的静态函数里没有使用make_shared来创建shared_ptr，例如像下面这样?

```
    static std::shared_ptr<ProcExecutor> Create() {
        return std::make_shared<ProcExecutor>();
    }
```

B站：精神浪子_码农